Project FRIDAY

A local, Jarvis-style second-brain assistant built on Obsidian + the Claude Agent SDK — chat, task & schedule optimization, RAG over your daily life, and a coursework engine for college: study guides, practice exams, and syllabus-aware planning. Implementation Plan v0.2

Prepared for: Ethan Soliven · Date: July 11, 2026 · Status: Draft for review

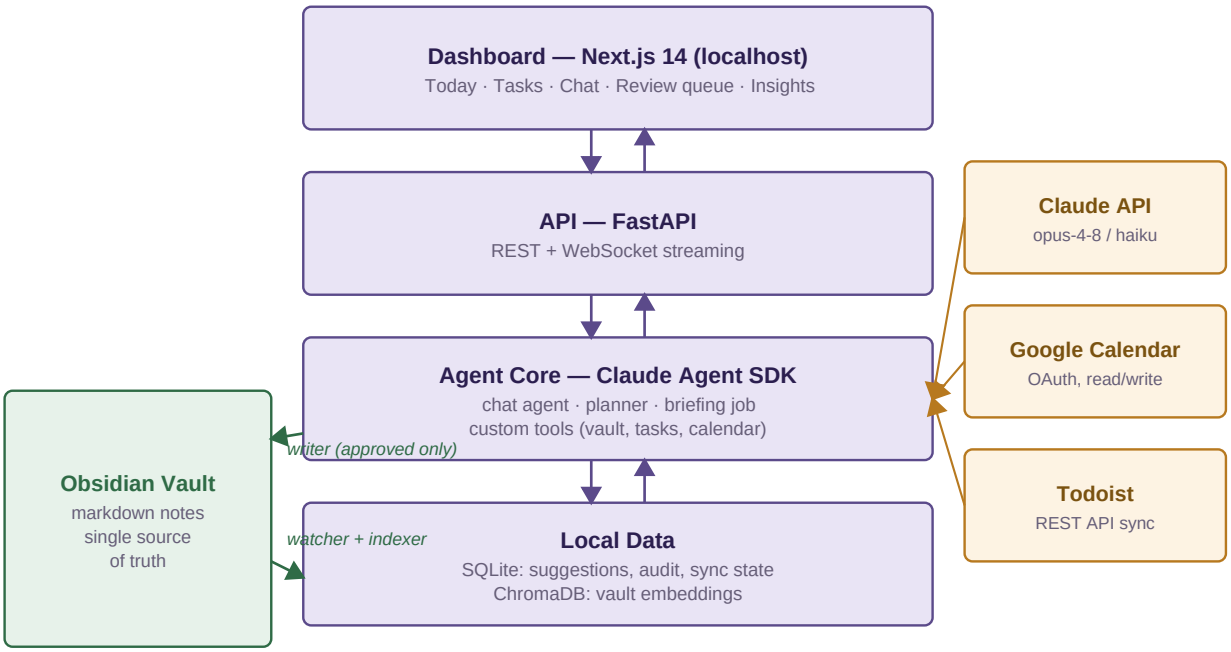
1. Executive Summary

FRIDAY turns an Obsidian vault into a queryable second brain with an assistant on top. A local web app (Next.js + FastAPI) provides **chat** grounded in your notes via **RAG**, and a **dashboard** that merges your Obsidian tasks, Google Calendar, and Todoist into one view and **optimizes your day**: the agent (Claude Agent SDK) proposes time blocks, task priorities, and note updates, and applies them only after your one-click approval. Because you're in college, FRIDAY doubles as a **study companion**: you drop lecture slides, readings, and syllabi into a course-materials folder; FRIDAY indexes them and generates study guides and practice exams grounded in *your* actual course content, and imports syllabus deadlines into your task system. The vault stays the single source of truth — everything FRIDAY learns and does is written back as plain markdown you own.

In scope (v1)	Out of scope (v1)
PARA-style vault template created from scratch	Mobile app (Obsidian mobile still syncs the vault)
Local RAG index over the vault (local embeddings)	Voice interface (phase-2 candidate)
Chat with note citations; morning briefing	Email triage (phase-2 candidate)
Task + calendar + Todoist merge and day planner	Multi-user / hosted deployment
Course hub: uploaded materials (PDF/PPTX/DOCX) indexed into RAG	Autonomous writes without review
Study-guide + practice-exam generation; syllabus deadline import	LMS integration (Canvas/Moodle API) — phase-2 candidate
Suggest-then-approve writes with audit trail	Doing your homework for you — FRIDAY drills you, it doesn't submit

2. System Architecture

Four local layers plus the Obsidian vault on one side and external services on the other. Runs via Docker Compose or two dev processes (`uvicorn` + `next dev`); no cloud hosting.



Everything runs on your machine; only agent prompts (with retrieved note excerpts) go to the Claude API.

Figure 1 — FRIDAY architecture. The vault is read continuously and written only through the approval gate.

- **Agent core:** Python + Claude Agent SDK. Default model `claude-opus-4-8` for planning/chat quality; `claude-haiku-4-5` for cheap summarization jobs. Adaptive thinking on; prompt caching on the stable system prompt + tool definitions.
- **Backend:** FastAPI — REST for dashboard data, WebSocket for streaming chat and live suggestion updates.
- **Frontend:** Next.js 14 (TypeScript, Tailwind). Notes open in Obsidian via `obsidian://open?path=...` deep links.
- **Storage:** SQLite (suggestions, audit log, connector sync state) + ChromaDB persistent directory (embeddings). Both live next to the vault in a `.friday/` folder — one directory to back up.

3. Vault Template (starting from scratch)

Since the vault is new, FRIDAY ships it as a template the installer creates on first run — PARA-inspired, tuned for machine readability:

```
vault/
  00-Inbox/            # quick capture, triaged by FRIDAY suggestions
  10-Daily/2026-07-11.md  # daily note: agenda, log, reflections, tasks
  15-Courses/          # one folder per course, e.g. 15-Courses/CS201/
    CS201/course.md    # course hub note: schedule, grading, prof, links
    CS201/notes/       # YOUR lecture & reading notes (markdown)
    CS201/materials/   # UPLOADS: slides, readings, syllabus (PDF/PPTX/DOCX)
    CS201/study/       # FRIDAY output: study guides, practice exams, results
  20-Projects/         # one note per active project (goal, next actions)
  30-Areas/           # health, finances, learning — ongoing responsibilities
  40-People/          # one note per person; FRIDAY links mentions
  50-Reference/       # evergreen knowledge notes
  99-Private/         # excluded from the RAG index and from Claude entirely
```

- **Tasks:** Obsidian Tasks plugin syntax — `- [ ] Renew passport [due: 2026-07-20] [prio: high] #areas/admin` (the plugin's emoji markers for due date and priority, shown here as text) — parsed by FRIDAY's task engine into due date, priority, and tag.
- **Frontmatter conventions:** every note carries `type`, `created`, `tags`; project notes add `status` and `deadline` — this metadata drives both RAG filters and the dashboard.
- **Course materials are drag-and-drop:** anything dropped into `materials/` (or uploaded via the dashboard's Study page) is picked up by the watcher and indexed — no manual import step.
- **Recommended community plugins:** Tasks, Dataview, Periodic Notes, Templater. FRIDAY reads their formats but never depends on them running — it parses the markdown directly.

4. RAG Pipeline — the Context of Your Daily Life

- **Watcher:** Python `watchdog` observes the vault; on create/modify/delete it re-indexes only the changed note (debounced ~2s). Full rebuild command for first run.
- **Document extraction:** non-markdown files in `materials/` pass through an extraction step first — `pdfplumber` for PDFs, `python-pptx` for slides, `python-docx` for docs — with **page/slide numbers kept as metadata** so study answers can cite 'Lecture 7, slide 12'.
- **Chunking:** markdown-aware — split on headings, target ~350 tokens with 50-token overlap; each chunk keeps metadata: path, title, heading, note date, tags, outgoing wikilinks, and course code for `15-Courses/` content (so retrieval can be scoped to one course).
- **Embeddings:** local `sentence-transformers` bge-small-en (CPU-friendly, free). Your note text is never sent anywhere for indexing.
- **Retrieval:** hybrid — vector similarity + BM25 keyword, fused, then **recency-weighted** (exponential decay on note date) because daily-life context favors this week over last year. Metadata filters (type, tag, date range) and one hop of **wikilink expansion**: if a retrieved chunk links to [[Mom]] or [[Project X]], those notes' summaries ride along.
- **Privacy boundary:** `99-Private/` and any note tagged `#no-ai` are excluded at index time. Retrieved excerpts *are* sent to the Claude API inside prompts — that is the one place vault text leaves your machine (see §9).

5. Agent Design (Claude Agent SDK)

One agent runtime, three entry points sharing the same tool belt: the **chat agent** (interactive, RAG-grounded), the **planner** (invoked from the dashboard or on schedule to lay out your day), and the **briefing job** (07:00 cron — compiles agenda, due tasks, and yesterday's loose ends into the daily note and dashboard).

Tool	What it does	Writes?
search_vault(query, filters)	Hybrid RAG retrieval with recency weighting and link expansion	No
read_note(path) / list_tasks(filter)	Full note content; parsed task list across vault + Todoist	No
get_calendar(range)	Google Calendar events for the range	No
propose_schedule(blocks)	Draft time blocks around fixed meetings (deep work, errands, breaks)	Queue only
propose_task_changes(changes)	Reprioritize, reschedule, or break down tasks	Queue only
propose_note_edit(path, patch)	Draft note edits — inbox triage, people-note updates, weekly review	Queue only
generate_study_guide(course, scope)	RAG over that course's notes + materials → structured guide saved to study/	Vault (study/)

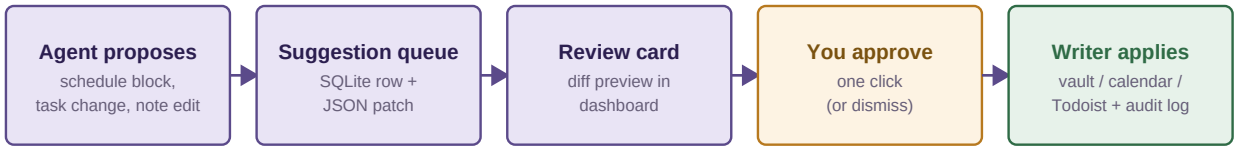
Tool	What it does	Writes?
generate_practice_exam(course, topics, n, difficulty)	Exam grounded in course materials, with answer key + page/slide citations	Vault (study/)
grade_attempt(exam, answers)	Scores a quiz-mode attempt; explains misses; logs weak topics for review	Vault (study/)
parse_syllabus(file)	Extracts exam dates, assignment deadlines, grading weights → proposed tasks	Queue only
apply_suggestion(id)	Execute an approved suggestion via the writer; append to audit log	Yes — gated

All `propose_*` tools use `strict: true` JSON schemas so patches are guaranteed parseable. The planner's system prompt encodes your preferences (class schedule, energy pattern, focus-block length) — stored as a note in `30-Areas/` so you can edit your own assistant's behavior in Obsidian. Study outputs are exempt from the approval gate: they only ever write new files under `study/`, never modify your notes.

College mode — how the coursework engine works

- **Semester setup:** create a course via the Study page → FRIDAY scaffolds `15-Courses/<CODE>/`, you drop the syllabus in `materials/` → `parse_syllabus` proposes every exam date and deadline as tasks with lead-time reminders (review before you approve).
- **Study guides:** scoped by unit or exam ('everything since midterm 1') — outline, key concepts with definitions from *your* slides, worked examples, and a 'what you haven't taken notes on' gap list comparing lecture materials against your `notes/` folder.
- **Practice exams:** question mix mirrors the course (from the syllabus grading section or your prompt); every question carries a citation to its source slide/page. Take them in the dashboard's quiz mode; `grade_attempt` scores you and appends weak topics to a per-course *review queue* the planner uses when scheduling study blocks — a lightweight spaced-repetition loop.
- **Exam-week planning:** the planner weighs syllabus deadlines and weak-topic lists, so 'plan my week' before finals produces study blocks per course, biased toward what you're weakest in.

6. The Suggest-then-Approve Loop



Nothing is written to your vault, calendar, or Todoist without step 4.

Figure 2 — Every mutation flows through the review gate; approvals are logged to an audit note in the vault.

- Suggestions are SQLite rows: type (schedule / task / note), a JSON patch, a human-readable rationale, and status (pending / applied / dismissed).
- The Review queue renders each as a card — calendar blocks on a mini-timeline, note edits as a text diff. Approve calls the writer; dismiss records why (fed back to the planner as preference signal).
- The writer is the **only** code path with write access to the vault, Google Calendar, and Todoist — one choke point to audit. Every apply appends a line to `10-Daily/<today>.md` under *## FRIDAY log*.

7. Dashboard Specification

Page	Contents
Today	Merged timeline (calendar events + approved/proposed blocks), top-3 tasks, morning briefing, quick capture box → 00-Inbox
Tasks	Unified board across vault + Todoist: overdue / today / this week / someday; bulk actions raise suggestions
Chat	Streaming conversation with FRIDAY; every claim cites source notes (click → opens in Obsidian); slash-commands: /plan, /review, /find
Study	Course cards (next deadline, weak topics); upload materials; generate study guide / practice exam; quiz mode with grading and citations
Review	Pending suggestion cards with diffs; approve / edit / dismiss; audit history
Insights	Task completion trends, overdue heatmap, calendar load vs. focus time, study streak + practice-exam score trends per course

8. Roadmap — 6 Phases (Claude Code-assisted)

Phase	Scope	Exit criteria
P0 — Foundation	Vault template generator; repo scaffold (FastAPI + Next.js); .friday/ storage; config	Fresh vault created; app boots and lists notes
P1 — RAG + chat	Watcher, chunker, local embeddings, ChromaDB, hybrid retrieval; PDF/PPTX/DOCX extraction for materials/; streaming chat with citations (read-only agent)	Ask about your notes OR a lecture deck; answers cite note/slide and deep-link
P1.5 — Coursework	Course scaffolding, Study page, generate_study_guide + generate_practice_exam + quiz mode + grade_attempt, parse_syllabus	Upload a real course's materials; generated exam is accurate and fully cited
P2 — Tasks + calendar	Task parser (Tasks syntax), Google Calendar OAuth (read), Todoist sync (read); Today + Tasks pages	One merged view of your real day
P3 — Planner + approvals	propose_* tools, suggestion queue, Review UI, writer with vault/calendar/Todoist write + audit log	Plan-my-day produces blocks you approve and see land in Google Calendar
P4 — Briefings + insights	07:00 briefing job into daily note + dashboard; Insights page; dismissal-feedback loop; polish	One week of daily use without manual intervention

9. Privacy, Cost & Risks

Privacy model — honest version

Indexing, embeddings, storage, and the UI are fully local. The exception: when the agent reasons, the prompt (your question + retrieved note excerpts + task/calendar snippets) goes to the Anthropic API over TLS. Mitigations: the `99-Private/` + `#no-ai` exclusions, per-request logging of what was sent (viewable in the dashboard), and API-tier data handling (Anthropic API data is not used for training). OAuth tokens (Google, Todoist) live in the OS keyring, never in the vault or repo.

Cost model — subscription-first

The agent authenticates with your existing **Claude subscription** (Claude Code login) rather than a pay-per-token API key, so agent usage draws from the plan you already pay for — **$0 marginal cost**. It shares the subscription's rate-limit windows with your own Claude Code sessions; FRIDAY's usage is light (briefings, chats, the occasional exam generation), so contention should be rare. An API key is the documented fallback for overflow.

Item	Estimate
Embeddings, storage, hosting, document extraction (all local)	$0
Agent (chat, planner, briefings, study tools) via Claude subscription auth	$0 marginal — uses existing plan quota
Optional API-key overflow (only if quota contention appears)	$0–10
Google Calendar / Todoist APIs	$0 (free tiers)
Total	**$0–10 / month**

Top risks and mitigations

Risk	Mitigation
Vault corruption from a bad write	Single writer path; JSON-patch previews; git-init the vault and auto-commit before every apply (free undo)
Second-brain habit doesn't stick	P4 briefing + quick-capture lower the friction; the tool must save time in week 1 or be re-scoped
RAG retrieves stale/wrong context	Recency weighting, citations on every answer so errors are visible, #no-ai escape hatch
Google OAuth setup friction	Desktop-app OAuth flow documented in P2; Todoist (simple token) lands first as fallback
Subscription quota contention with your own coding sessions	Prompt caching + lean retrieval keep usage small; scheduled jobs movable to an API key as overflow
Generated exams contain errors (hallucinated content)	Every question must carry a source citation to a slide/page; uncited questions are dropped; quiz mode shows the source after answering
Academic integrity	Scope is studying — guides, drills, planning. FRIDAY does not write submissions; that boundary is stated in the planner prompt

---

FRIDAY is a personal productivity tool. Your vault remains plain markdown on your disk — if you delete FRIDAY tomorrow, your second brain stays.